

EE446-TinyML Lab 7

Author: Sparsh Dadhich

Github: <https://github.com/sdadhich04/EE-446-TinyML-/tree/main/Lab%208>

Edge Impulse: <https://studio.edgeimpulse.com/public/1013712/live>

Part 1

```
Timing: DSP 80 ms, inference 533 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 533 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.011719
  lift: 0.988281
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 580 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.902344
  lift: 0.097656
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 569 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 568 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
```

```

Timing: DSP 84 ms, inference 568 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 540 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 541 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 83 ms, inference 552 us, anomaly 0 ms, postprocessing 67 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 84 ms, inference 551 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 79 ms, inference 551 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...

```

Part 2

1. The ensemble activity recognition model takes a 100×6 IMU data window and first flattens it into a 600-dimensional feature vector. This same input is then processed through three parallel branches that use different preprocessing strategies: raw data, standard-scaled data, and min-max scaled data. Each branch contains an encoder network with a $600 \rightarrow 64 \rightarrow 32$ architecture, where the first hidden layer uses ReLU activation and the final encoder layer produces a 32-dimensional latent representation. The output of each encoder is passed into a classifier consisting of a $32 \rightarrow 20 \rightarrow 12$ architecture, with a ReLU hidden layer and a 12-class softmax output layer. This produces three separate probability vectors, one from each preprocessing branch. The three 12-class probability outputs are then concatenated into a single 36-dimensional vector and passed into a stacked meta-classifier with a $36 \rightarrow 24 \rightarrow 12$ architecture. The meta-classifier uses a ReLU hidden layer followed by a 12-class softmax output layer to generate the final activity prediction. Overall, the system contains three encoder models, three branch classifiers, and one stacked meta-classifier, resulting in a total of seven trained models working together as an ensemble. The purpose of this design is to allow the model to leverage different

feature distributions created by the three preprocessing methods and combine their predictions through a higher-level classifier to improve overall classification performance and robustness.

2. The model compression pipeline combines magnitude-based pruning, Quantization Aware Training (QAT), and full integer quantization to produce a TinyML-friendly deployment model. Pruning is first applied using TensorFlow Model Optimization's PolynomialDecay schedule, which gradually increases sparsity from 10% to 80% over 500 training steps. Rather than removing many weights at once, the pruning schedule progressively forces low-magnitude weights to zero, allowing the network to adapt during training while minimizing accuracy loss. After pruning is complete, the pruning wrappers are removed using `strip_pruning()`, which eliminates the additional masks and bookkeeping variables while preserving the learned sparse weight structure. This step is necessary before further optimization and deployment because it reduces unnecessary overhead in the exported model. Quantization Aware Training is then applied to simulate low-precision arithmetic during training so the model can learn to operate under the constraints of int8 quantization. A custom `MaskEnforcerCallback` is used throughout QAT to ensure that previously pruned weights remain exactly zero, preventing quantization fine-tuning from restoring weights that were intentionally removed during pruning. After training, the model is converted to a fully int8 TensorFlow Lite model using a representative dataset. The representative dataset is important because it allows the converter to observe real activation ranges and calculate appropriate quantization scales and zero-points for each tensor. Combining pruning with int8 quantization significantly reduces model size, memory usage, and computational requirements, making the final model much more suitable for deployment on resource-constrained microcontrollers used in TinyML applications while maintaining strong classification accuracy.

3.

```
-----
Final prediction: Standing still
Class index: 0
Confidence score: 0.3867
-----

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0039, 0.0078, 0.0039, 0.0000, 0.0078, 0.0000, 0.0039, 0.0000, 0.9766
Standard-scaled model scores: 0.0000, 0.0273, 0.0000, 0.0000, 0.3477, 0.0000, 0.0000, 0.6172, 0.0000, 0.0078, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.8164, 0.0000, 0.1836, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.3984, 0.0430, 0.2148, 0.0117, 0.0820, 0.0078, 0.0195, 0.0039, 0.0078, 0.0117, 0.0078, 0.1914
-----

Final prediction: Standing still
Class index: 0
Confidence score: 0.3984
-----

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0000, 0.0039, 0.0078, 0.0039, 0.0000, 0.0078, 0.0000, 0.0039, 0.0000, 0.9766
Standard-scaled model scores: 0.0000, 0.0273, 0.0000, 0.0000, 0.3750, 0.0000, 0.0000, 0.5898, 0.0000, 0.0078, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.7891, 0.0000, 0.2070, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.3867, 0.0430, 0.1992, 0.0117, 0.1055, 0.0078, 0.0195, 0.0039, 0.0078, 0.0117, 0.0078, 0.1992
-----

Final prediction: Standing still
Class index: 0
Confidence score: 0.3867
-----
```

The deployment results demonstrate that the complete TinyML ensemble pipeline was successfully executed on-device and was able to perform real-time activity classification using live IMU data. For each inference cycle, the system collected a full window of 100 sensor samples before running the three branch classifiers and the stacked meta-classifier. The raw classifier consistently produced a very strong prediction for class 11 with confidence values around 0.98, while the standard-scaled and min-max-scaled classifiers produced different predictions with varying confidence levels. This behavior is expected because each branch processes the same sensor data using a different feature representation and therefore learns different decision boundaries. The outputs from all three classifiers were combined by the stacked meta-classifier, which generated the final prediction. In the example shown, the ensemble repeatedly predicted "Standing still" (class 0) with confidence scores of approximately 0.39. Although the confidence score is lower than the individual branch predictions, this is expected because the meta-classifier is combining conflicting evidence from multiple models rather than relying on a single highly confident prediction. The fact that the prediction remained stable across consecutive inference windows indicates that the ensemble was operating consistently and that the complete preprocessing, encoder, classifier, and meta-classification pipeline was functioning correctly on the embedded device. These results demonstrate successful deployment of the pruned and quantized TinyML ensemble model and confirm that real-time activity recognition can be performed within the computational and memory constraints of the target hardware.

4. Yes, I observed some unexpected prediction behavior during deployment. Even when the Arduino board was moved in different ways, the ensemble model frequently predicted the same class, "Standing still," across multiple inference windows. This behavior is not uncommon when deploying a model trained on a dataset that differs from the live sensor data collected by the hardware. One possible cause is that the original mHealth training dataset was collected using different sensors, sensor placements, sampling conditions, and participant motion patterns than those available on the Arduino Nano 33 BLE Sense. Differences in accelerometer sensitivity, noise characteristics, and sampling accuracy can cause the live data distribution to differ significantly from the training data distribution. Sensor orientation is another important factor because the model was trained assuming specific axis directions, and rotating the board changes how motion is represented across the X, Y, and Z axes. In addition, differences in units, scaling, or preprocessing can affect performance if the live data is not transformed exactly the same way as the training data. Even small mismatches in standardization or min-max scaling can cause the model inputs to fall outside the range it learned during training. Finally, the motion patterns performed during testing may not closely match the activities represented in the training dataset, causing the ensemble to default toward a familiar class that appears most similar to the incoming sensor data. These factors can lead to reduced confidence, class bias, and repeated predictions even when different motions are performed on the deployed device.